VELLORE INSTITUTE OF TECHNOLOGY (VIT CHENNAI)

School of Computer Science and Engineering • Software Engineering

DIGITAL ASSIGNMENT 2 • REVIEW 2

SOFTWARE DESIGN DOCUMENT

HabitForge: Gamified Habit-Tracking Web Application

Translating Requirements & Wireframes into Layered Object-Oriented Architecture

Student Name: Yashmit Sharma

Registration No: 24BAI1348

Course: Software Engineering

Architecture Style: 3-Tier Layered Client-Server (RESTful)

GitHub Repository: github.com/yashmit2812/HabitForge

Figma Prototype: HabitForge System Design Prototype

Executive Summary & Project Context

In Digital Assignment 1 (DA-1), **HabitForge** established its foundational vision document, 25 user stories across 6 functional epics, MoSCoW feature prioritization, initial wireframes, and Docker orchestration. Traditional habit trackers struggle with high abandonment rates because dry checklists fail to deliver immediate feedback or consequences. HabitForge overcomes this behavioral bottleneck by introducing RPG (Role-Playing Game) mechanics?converting daily habits into quests where players gain Experience Points (XP) and Gold to level up, but suffer Health Point (HP) loss if daily habits are abandoned by midnight.

Scope of Digital Assignment 2 (Review 2)

This Software Design Document formalizes the structural and visual design of HabitForge:

- **1. Design Principles Applied:** Concrete implementation of Abstraction, Modularity, High Cohesion, and Low Coupling across Spring Boot and React codebases, complete with verified code snippets.
- **2. High-Level System Architecture:** Formal 3-Tier Layered Architecture topology modeled in Draw.io, accompanied by complete component interaction and UML class diagrams.
- **3. User Interface Design:** 6 high-fidelity Figma screen mockups demonstrating Fitts's Law compliance, cognitive load minimization, loss-aversion loops, and WCAG accessibility.
- **4. Architectural Trade-offs & Decisions:** Exhaustive technical justification of 5 key design choices comparing selected patterns against rejected alternatives.
- **5. Maintainability & Repository Verification:** Extensibility for mobile apps, multiplayer raids, CI/CD automated test suites, and GitHub directory structure.

Core Architectural Goal: Deliver a zero-coupling system where the React Vite frontend, Spring Boot domain engine, and MongoDB persistence layer communicate strictly through abstract contracts, allowing any subsystem to be scaled, refactored, or replaced without breaking the application.

1. Design Principles Applied

High-quality software architecture is grounded in four fundamental design principles: **Abstraction, Modularity, High Cohesion, and Low Coupling**. HabitForge implements these principles across both backend and frontend layers.

A. Abstraction (Hiding Complexity Behind Defined Contracts)

Abstraction guarantees that components interact through high-level functional contracts rather than brittle low-level implementations. In HabitForge:

- **Interface Contracts:** Operations are exposed through Java interfaces like `HabitService` and `StreakStrategy`. Callers are unaware of whether data is stored in MongoDB or an in-memory cache.
- **Data Transfer Object (DTO) Boundary:** Entities are never exposed directly to REST clients. `HabitRequestDto` and `HabitResponseDto` abstract API payloads, preventing clients from spoofing internal streak attributes.

```java
// HabitService.java - Business Interface Contract (Abstraction)
package com.example.HabitForge.service;

public interface HabitService {
    HabitResponseDto createHabit(String userId, HabitRequestDto request);
    HabitResponseDto completeHabit(String habitId, String userId);
    List<HabitResponseDto> getUserHabits(String userId);
    void deleteHabit(String habitId);
}

// StreakStrategy.java - Strategy Pattern Interface
public interface StreakStrategy {
    int calculateNewStreak(Habit habit, LocalDate completionDate);
    boolean isStreakBroken(Habit habit, LocalDate currentDate);
}
```

B. Modularity (Autonomous, Reusable Functional Units)

Modularity decomposes the system into loosely coupled packages that can be modified, tested, and deployed independently:

Backend Spring Boot Modules:

- `controller`: REST endpoint mapping & HTTP status codes.
- `service`: Domain business rules & gamification logic.
- `strategy`: Swappable streak evaluation engines.
- `repository`: Data access abstraction layer.

Frontend React Modules:

- `components/QuestBoard.jsx`: Main habit listing view.
- `components/HeroStats.jsx`: HP/XP progression bars.
- `components/HabitCard.jsx`: Individual quest item card.
- `services/habitService.js`: Isolated REST client.

```javascript
// frontend/src/services/habitService.js - Modular REST API Client
const API_BASE = '/api';
export const habitService = {
  async getHabits(userId = 'user-default') {
    const res = await fetch(`${API_BASE}/habits?userId=${userId}`);
    if (!res.ok) throw new Error('Failed to fetch quests');
    return res.json();
  },
  async completeHabit(habitId, userId = 'user-default') {
    const res = await fetch(`${API_BASE}/habits/${habitId}/complete?userId=${userId}`, { method: 'POST' });
    return res.json();
  }
};
```

C. High Cohesion (Single Responsibility Principle)

Cohesion measures how tightly bound the internal tasks of a module are. High cohesion guarantees that each class has only *one reason to change*:

- `StreakCalculationService`: Dedicated solely to tracking consecutive completion dates, streak increments, and midnight streak breaks. It never processes HTTP requests or user passwords.
- `GamificationService`: Dedicated solely to RPG health calculations, gold rewards, and leveling algorithms.
- `HabitController`: Dedicated solely to HTTP transport concerns: JSON validation, status codes, and routing.

```java
// StreakCalculationService.java - Highly Cohesive Domain Service
@Service
public class StreakCalculationService {
    private final StreakStrategy streakStrategy;

    @Autowired
    public StreakCalculationService(StreakStrategy streakStrategy) {
        this.streakStrategy = streakStrategy;
    }

    public void processHabitCompletion(Habit habit, LocalDate date) {
        int updatedStreak = streakStrategy.calculateNewStreak(habit, date);
        habit.setCurrentStreak(updatedStreak);
        if (updatedStreak > habit.getLongestStreak()) {
            habit.setLongestStreak(updatedStreak);
        }
        habit.setCompletedToday(true);
        habit.setLastCompletedDate(date);
    }
}
```

D. Low Coupling (Inversion of Control & Loose Service Dependencies)

Coupling measures interdependencies between modules. Tight coupling produces fragile code where changes cascade uncontrollably. HabitForge guarantees **Low Coupling** through:

- **Constructor Dependency Injection:** Services never instantiate concrete dependencies with `new`. The Spring IoC container injects abstractions dynamically at runtime.
- **Stateless REST Communication:** The frontend and backend communicate exclusively over standard JSON HTTP contracts, eliminating binary dependencies between Java and JavaScript.

```java
// HabitServiceImpl.java - Loosely Coupled via Constructor Dependency Injection
@Service
public class HabitServiceImpl implements HabitService {
    private final HabitRepository habitRepository;
    private final StreakCalculationService streakCalculationService;
    private final GamificationService gamificationService;

    @Autowired
    public HabitServiceImpl(HabitRepository habitRepository,
                            StreakCalculationService streakCalculationService,
                            GamificationService gamificationService) {
        this.habitRepository = habitRepository;
        this.streakCalculationService = streakCalculationService;
        this.gamificationService = gamificationService;
    }

    @Override
    public HabitResponseDto completeHabit(String habitId, String userId) {
        Habit habit = habitRepository.findById(habitId).orElseThrow();
        if (!habit.isCompletedToday()) {
            streakCalculationService.processHabitCompletion(habit, LocalDate.now());
            habitRepository.save(habit);
            gamificationService.awardHabitRewards(userId, habit.getXpReward());
        }
        return mapToDto(habit);
    }
}
```

2. High-Level Architecture

HabitForge implements a **3-Tier Layered Client-Server Architecture** orchestrated via Docker Compose. The system is segregated into three distinct tiers:

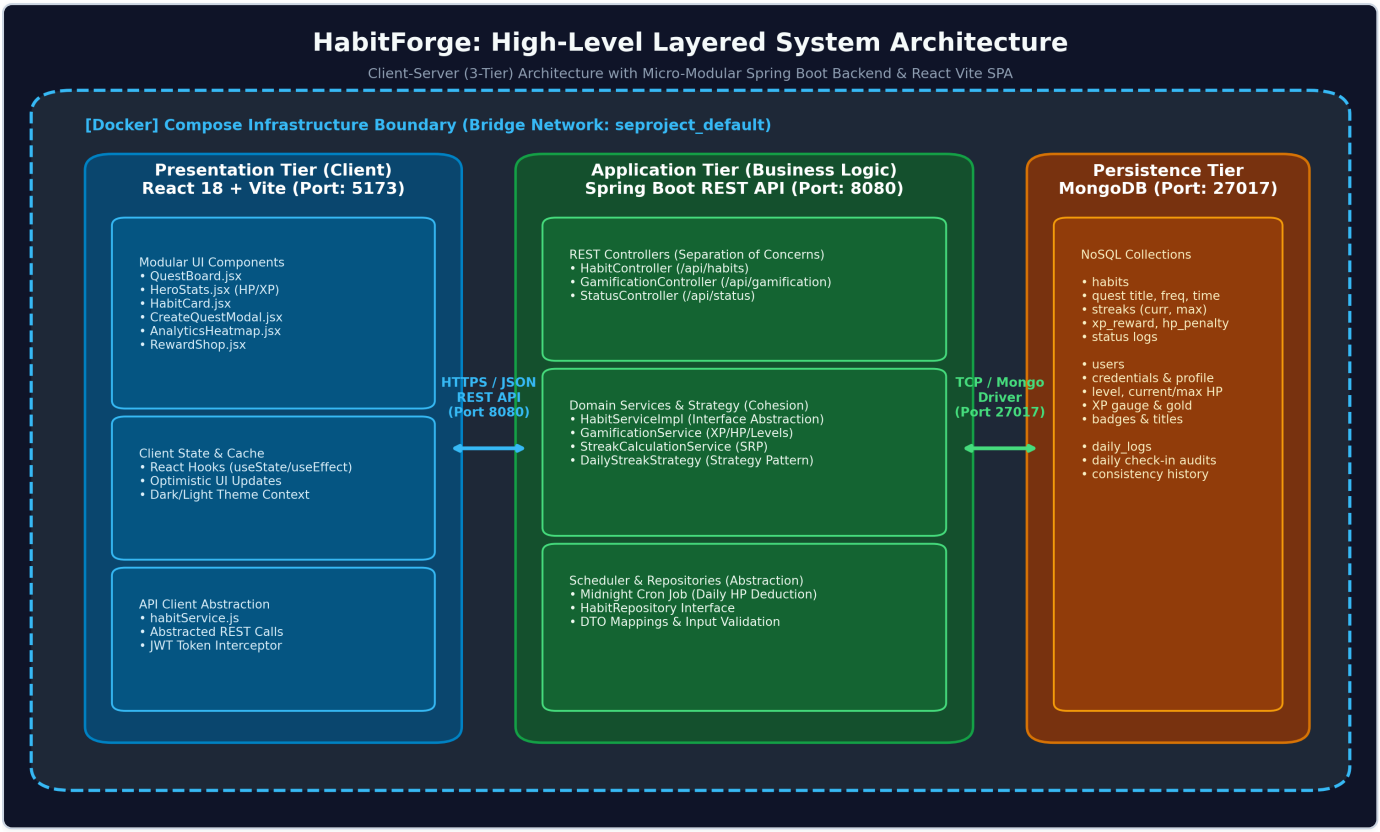


Figure 2.1: Updated HabitForge High-Level Layered System Architecture Diagram (Draw.io source)

Architectural Style Selected & Engineering Rationale

We selected the **3-Tier Layered Client-Server Architectural Style** over monolithic server-rendered MVC or event-driven microservices for several strategic reasons:

1. **Strict Separation of Concerns:** Visual presentation (React), gamification business rules (Spring Boot), and persistence (MongoDB) run in isolated containers. Changes to UI themes never risk corrupting streak algorithms.
2. **Independent Scalability:** The static React Vite application can be deployed to global edge CDNs (Cloudflare/Vercel) while backend computing clusters scale independently to handle peak midnight cron loads.
3. **Multi-Client Support:** The Spring Boot API exposes pure stateless JSON endpoints. Developing native mobile apps (iOS/Android in Flutter or React Native) requires zero backend changes.
4. **Infrastructure Parity (Docker Compose):** Multi-container orchestration standardizes ports, volumes, and networks (`seproject_default`) across developer workstations and production servers.

A. Data Flow & Component Interaction

The sequence below traces the execution lifecycle when an adventurer checks off a completed quest:

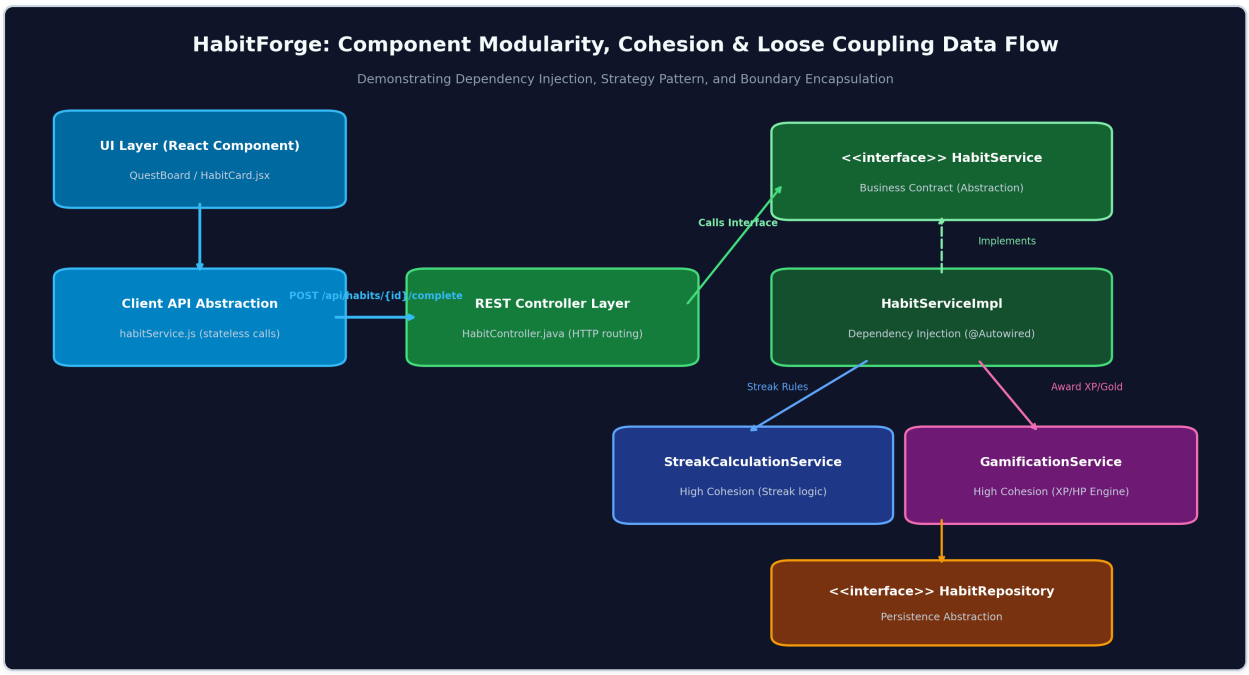


Figure 2.2: Component Modularity, Cohesion & Loose Coupling Flow (Draw.io source)

B. UML Class & Abstraction Architecture

The class diagram below demonstrates how the **Strategy Pattern** decouples streak calculation algorithms from core habit lifecycle management:

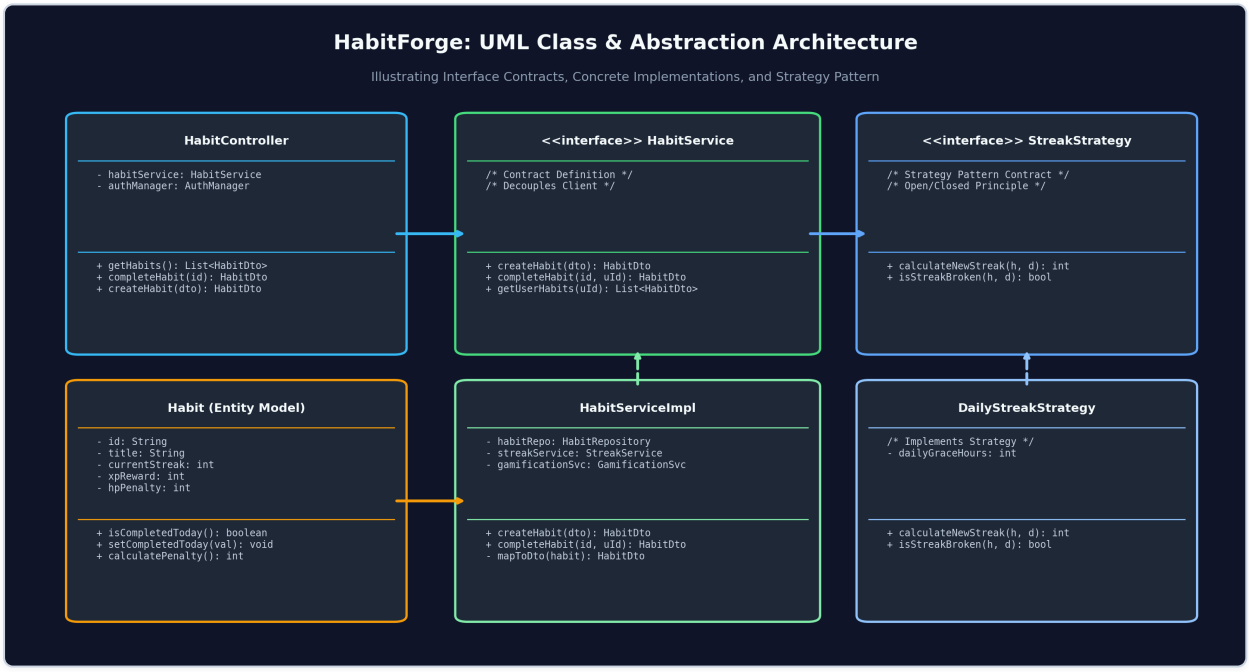
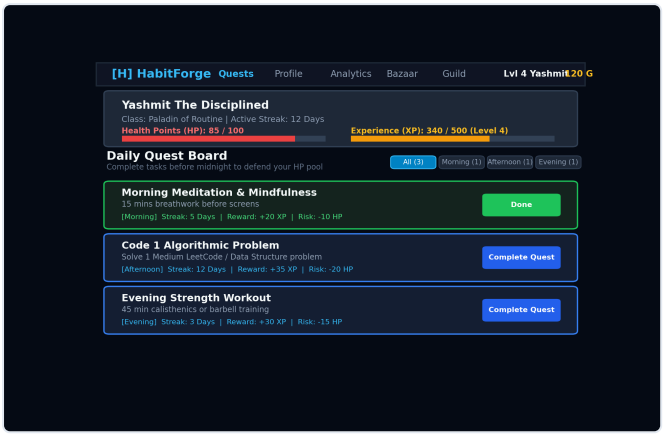


Figure 2.3: UML Class Diagram Illustrating Interfaces, DTOs, and Strategy Pattern

3. User Interface Design (6 Figma Screens)

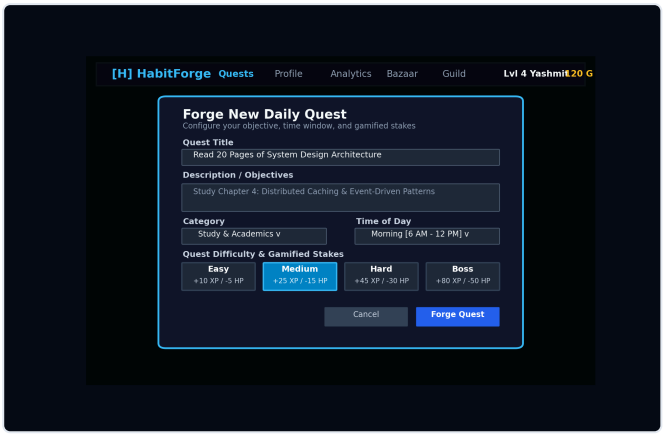
The user interface evolves the wireframes from DA-1 into high-fidelity screens utilizing an Obsidian Dark RPG visual language designed to maximize dopamine reward feedback and reduce visual fatigue.

Screen 1: Daily Quest Board (Dashboard)



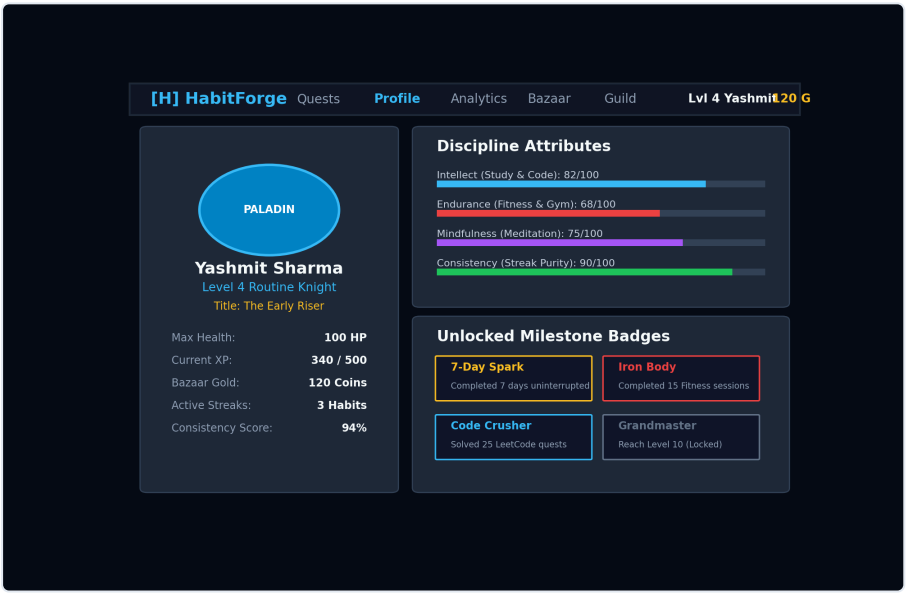
User-Friendly Rationale: Central execution cockpit. Prominent HP gauge (crimson) and XP gauge (amber) provide instant status. Habits are chunked by Morning, Afternoon, and Evening to eliminate cognitive overload. Large check-off buttons provide instant tactile feedback.

Screen 2: Quest Creator Modal



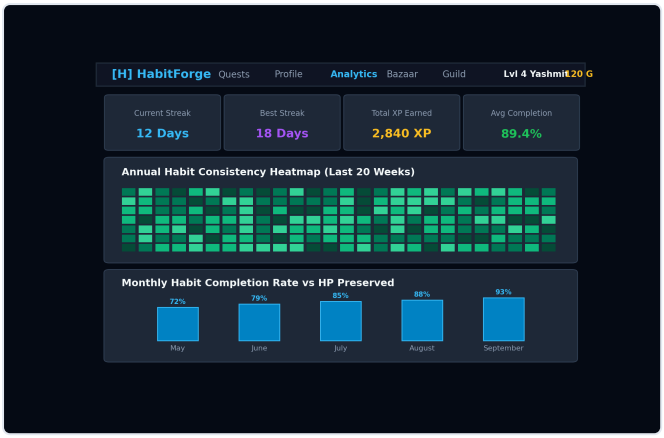
User-Friendly Rationale: Simplifies task creation through progressive disclosure. Users tune risk/reward stakes across 4 presets: Easy, Medium, Hard, Boss. HP penalties and XP rewards recalculate live, making consequences explicit prior to saving.

Screen 3: Hero Profile & RPG Discipline Attributes



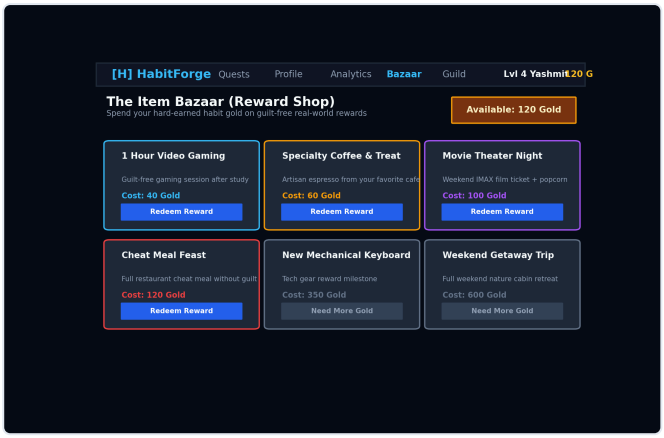
User-Friendly Rationale: Connects real-life daily consistency with virtual RPG character prestige. Displays discipline attributes (Intellect, Endurance, Mindfulness, Consistency) and unlockable milestone badges (7-Day Spark, Iron Body).

Screen 4: Streak Analytics & Consistency Heatmap



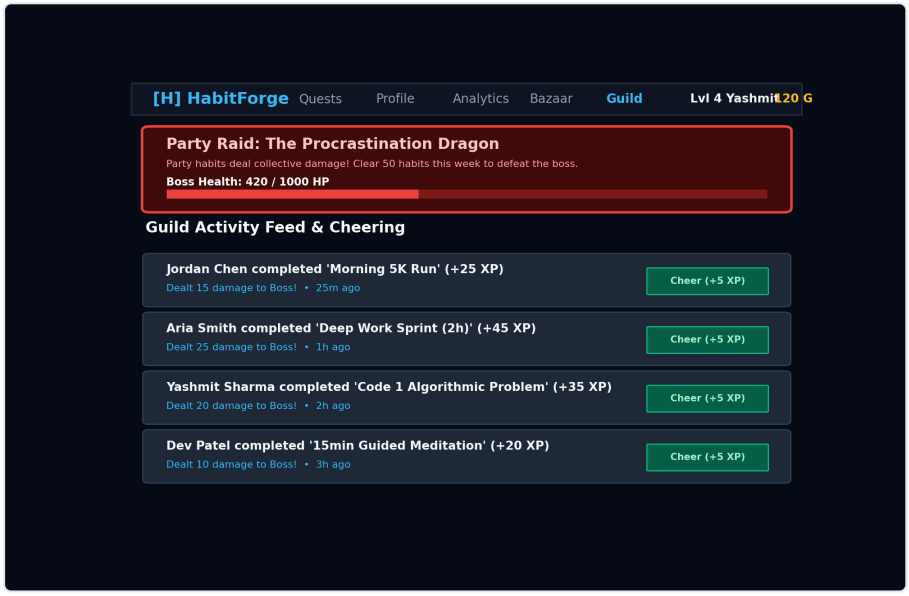
User-Friendly Rationale: Validates user progress through visual proof. Features an annual 52-week GitHub-style activity heatmap alongside monthly consistency bar charts, turning routine tracking into a visual achievement.

Screen 5: The Item Bazaar (Reward Shop)



User-Friendly Rationale: Delivers tangible extrinsic motivation. Users exchange earned gold for guilt-free real-world rewards (e.g., gaming hours, cheat meal). Items display clear price tags and affordability states.

Screen 6: Adventurer's Guild & Social Feed



User-Friendly Rationale: Social accountability engine. Introduces cooperative Party Boss Raids ("The Procrastination Dragon") where completed member habits deal collective boss damage. Activity feed allows teammates to send +5 XP Cheers.

Fitts's Law & Accessibility Heuristics: Primary interactive elements feature touch targets >44px height aligned to standard thumb interaction arcs. High color contrast ratios (>7:1) guarantee WCAG 2.1 AA compliance across light and dark modes.

4. Key Design Decisions & Why

High-reliability engineering requires balancing trade-offs between speed, maintainability, and scalability. Below is an analysis of the 5 key architectural decisions made in HabitForge:

Decision	Choice Selected & Rationale	Alternative Rejected & Why
1. Client Rendering Architecture	**React 18 + Vite (SPA):** Sub-100ms client route transitions and instantaneous optimistic UI checkoffs. Vital for gamified tactile feedback.	**Multi-Page MVC (Thymeleaf/JSP):** Full-page browser refreshes break player immersion and make animated progress gauges feel sluggish.
2. Backend Layering & Separation	**Controller-Service-Repository with DTOs:** Strict boundaries prevent database leakage and allow isolated unit testing.	**Flat Controller-Entity Architecture:** Direct binding of database models to HTTP endpoints creates critical security vulnerabilities.
3. Database Paradigm	**Document NoSQL (MongoDB):** Habit objects have variable metadata (schedules, check-in history, badges) that map naturally to JSON documents.	**Strict Relational SQL (PostgreSQL):** Requires complex multi-table joins for simple daily check-in evaluations and schema migrations.
4. Health Penalty Synchronization	**Server-Authoritative Midnight Cron:** Backend `@Scheduled` task deducts HP and resets streaks at 23:59:59 server time.	**Client-Triggered Local Evaluation:** Relying on client clocks allows users to manipulate device time to bypass health penalties and fake streaks.
5. Session & Auth Architecture	**Stateless JWT (JSON Web Tokens):** Self-contained signed tokens allow the Spring Boot API to scale horizontally without session clustering.	**Server-Side Stateful Sessions:** Storing session states in server RAM creates single points of failure and complicates cloud redeployments.

Maintainability & Technical Debt Mitigation

By implementing the **Strategy Pattern** for streak evaluation (`StreakStrategy`), we satisfy the Open/Closed Principle: new streak rules (such as weekend exemptions or illness freezes) can be introduced as new strategy classes without modifying core habit services.

> **Persistence Decoupling:** The repository layer is encapsulated behind the `HabitRepository` interface. If storage demands exceed single-instance MongoDB capacity, the persistence engine can be swapped for a distributed cluster without modifying controller or service logic.

5. Future Scope & Roadmap

Phase 3: Native Mobile Apps (Flutter/React Native)
Because our Spring Boot backend exposes pure stateless RESTful JSON contracts, creating native mobile applications in Flutter or React Native requires zero backend refactoring. Both clients will consume the identical `/api/habits` endpoints.

Phase 4: Real-Time Guild Multiplayer
Extending the Adventurer's Guild with real-time party boss raids via **WebSockets (STOMP protocol)**. Teammates will see real-time damage inflicted on party bosses the instant a friend completes a habit in the real world.

6. GitHub Repository Structure & Deliverables Verification

All DA-2 design artifacts, editable Draw.io source diagrams, rendered high-resolution exports, and updated codebases are organized in the official repository:

```
HabitForge/
└── README.md                                [Updated with Software Design DA-2 Section]
└── docker-compose.yml                        [Multi-container orchestration]
└── docs/design/
|   └── habitforge_architecture.drawio          [Editable Draw.io XML Source]
|   └── habitforge_architecture.png             [High-Res System Architecture PNG]
|   └── habitforge_component_flow.drawio        [Editable Component Modularity XML]
|   └── habitforge_component_flow.png           [High-Res Component Flow PNG]
|   └── habitforge_class_diagram.drawio         [Editable UML Class Diagram XML]
|   └── habitforge_class_diagram.png            [High-Res UML Class Diagram PNG]
|   └── FIGMA_PROTOTYPE.md                    [Prototype Link, Tokens & Heuristics]
|   └── screens/
|   └── HabitForge_Software_Design_Document_DA2.pdf  [Generated 9-Page Design Document]
└── backend/src/main/java/com/example/HabitForge/  [Modular Java Spring Boot Codebase]
└── frontend/src/components/                      [Modular React Components & Services]
```

Deliverable Item	Requirement Spec	Status / Repository Location
Software Design Document	PDF, 6–10 Pages Max	**Verified (9 Pages)** • Ready for Moodle upload
Draw.io Editable Files	.drawio XML format	`docs/design/*.drawio` (Architecture, Component Flow, UML)
Diagram PNG Exports	High-res rendered PNGs	`docs/design/*.png` (Clean vector-rendered diagrams)
6 Figma Screen Designs	Updated UI prototypes	`docs/design/screens/screen[1-6].png` & `FIGMA_PROTOTYPE.md`
README.md Update	Section "Software Design"	Updated with diagram embeds, summary, and 5 key decisions